ONE-WAVE FRAMEWORK

Book 1 — Micro

Chapter 1: Persistent Modes / What Standard Physics Calls Particles

Version: 3.0 Date: July 1, 2026 Class: B — Applied Layer Spine: Gray / 2D / 3D / Mathematics / Predictions / Yellow Audit / Future Work / Closing Thoughts

Dependencies: A-112 Persistent Mode, A-108 Local Stability, C-308 Spin-half, B-206 Four Views, C-318 Mass-Effect Mechanism Resolution, A-115 Unified Compression Field Status: GREEN

Gray — Standard Model Reference

In the Standard Model, particles are discrete irreducible fundamental units. An electron is a point particle with no internal structure, assigned mass, charge, and spin. A quark is a sub-constituent of protons and neutrons. Particles interact via force-carrier particles: photons, gluons, W/Z bosons. Mass is assigned via the Higgs mechanism — particles acquire mass by coupling to the Higgs field. Spin is an intrinsic quantum property with no classical analog. The particle ontology assumes objects exist first and fields second.

Standard Model strength: extremely precise predictions for particle interactions. Standard Model limitation: does not explain why particles have the masses they do, why there are three generations, or what mass fundamentally is.

2D One-Wave Interpretation

In a 2D field there are no particles. There are only regions where the field psi departs persistently from ground.

A region of persistent displacement that oscillates and returns, oscillates and returns, without collapsing — that is an excitation point. It is not a thing. It is a stable pattern.

In 2D: imagine a standing ripple on a water surface. The ripple is not made of anything special. It is just the surface doing something locally that it is not doing elsewhere. The ripple has no inside. It has no boundary made of special material. It is just the surface behaving differently at that location.

What we call a particle is the 2D projection of a 3D field pattern. The 2D view shows us the excitation but loses the full structure. This is why the 2D view is a tool, not the framework (A-11 Projection).

3D One-Wave Interpretation

In 3D, what are called particles are Persistent Modes of the superfluid lattice field psi.

The full chain from ground to excitation:

Ground (A-01) -> Displacement (A-02) -> Differential (A-03) -> Gradient (A-04) -> Restoring Response (A-05) -> Bounded Motion (A-06) -> Inertial Memory (A-07) -> Oscillation (A-08) -> Recursion (A-09) -> Persistent Mode (A-10) -> Excitation

An excitation is a non-ground-state pattern that survives recursive update without collapsing. The excitation has no boundary made of special material. It has no interior made of smaller things. It is the field doing something stable.

There are no particles. There are excitations.

The field is the superfluid lattice — a medium that supports wave propagation, has resistance (gamma), coupling (beta), and a reference ground (psi_0). Everything that exists is the field doing something locally different from ground.

Spin-half follows from the four views boundary roll-off (B-06b): When the mode presses against its boundary, surface tension holds it, then it rolls off. That roll-off produces the 4*pi closure: psi(theta + 2*pi) = -psi(theta) psi(theta + 4*pi) = psi(theta) This is spin-half. It emerges from the geometry. It is not assigned.

Propagation kinematics and Mass Effect are separate. C-309 constrains transport; C-318 calculates the response of the complete bounded recurrence. An earlier attempt to combine those jobs was a misunderstanding and has been removed.

The active One-Wave mechanism does not begin with a scalar-field mass gap. C-318 treats the measured Mass Effect as the resistance produced when the complete bounded recurrence must be carried and rebuilt relative to Ground. The internal knot, electrical shell, Mirror-Gate relation, Boundary-Tension Weave, and their cross-couplings all participate.

Charge follows from internal/external pressure asymmetry (B-06b, C-311): The roll-off at the mode boundary produces a pressure cushion. That cushion is the electric shell — the charge. The rotation of that cushion is magnetism. Charge is not assigned. It is the pressure geometry of the boundary.

Mathematics

Persistent Mode stability criterion (A-10): ||psi_{n+k} - psi_n|| < epsilon

Full update rule (A-09): psi_i^{n+1} = psi_i^n + (1-gamma)(psi_i^n - psi_i^{n-1}) + beta_i(<psi_j^n> - psi_i^n)

Spin-half closure (C-08, B-06b): psi(theta + 2*pi) = -psi(theta) psi(theta + 4*pi) = psi(theta)

Boundary roll-off excess energy (B-06b): DeltaE = E_mode - E_s DeltaE = E_mode - sigma * A_s

Roll-off condition: DeltaE > 0 => roll-off fires => spin tendency S_roll != 0 => pressure cushion P_c = beta * DeltaE / V_c => pressure pulse propagates outward

Propagation ceiling (C-309):

$$c_{\text{lat}} = v_{\text{max}} = \sqrt{\beta_{\text{max}}}\,\frac{\Delta x}{\Delta t}$$

This constrains traveling modes but does not generate mass.

Four-interaction Mass Effect (C-318):

$$\overline{E}_4(\mathbf{v}) = \overline{E}_4(0) + \frac{1}{2} v_i \mathcal{M}_{ij} v_j + \cdots$$

$$\mathcal{M}_{ij} = \left.\frac{\partial^2 \overline{E}_4}{\partial v_i \partial v_j}\right|_{\mathbf{v}=0}, \qquad m_{\text{eff}} = \frac{1}{3}\,\mathrm{Tr}\,\mathcal{M}$$

for an isotropic lowest mode.

Predictions

1. No truly structureless point particles. All excitations have minimum lattice-scale extent — a floor set by the lattice step size Delta_x. There is no mathematical point particle in One-Wave.
2. Spin-half behavior follows from 4*pi closure (B-06b, C-08). No SU(2) imported externally. It emerges from the boundary roll-off geometry.

3. The Mass Effect requires a stable four-interaction recurrence with nonzero carried-pattern resistance (C-318). Prediction: a completed model must produce $\mathcal{M}_{ij} > 0$ for bounded material modes while preserving a traveling light mode with zero rest Mass Effect under the same update law.
4. Charge is a pressure geometry property (B-06b, C-311). Prediction: charge and magnetic moment are related by the same roll-off geometry. No separate mechanism needed.
5. The unified field provides the Boundary-Resistance View associated with measured Mass Effect (A-115). The approximately 125 GeV collider measurement remains the separate C-322 Mirror-Gate boundary-response anchor. It is not the generic Mass Effect and cannot be inserted into the mass derivation by hand.

Yellow Audit

- Mode type classification deferred (A-10)
- Charge emergence from roll-off pressure cushion not yet formally completed (B-06b)
- Spin emergence from boundary asymmetry not yet formally proved (B-06b)
- Whether spin, mass, charge always co-emerge from same roll-off event unresolved
- The four-interaction response matrix has not yet been derived from a stable 3D recurrence or used to produce a measured Mass Effect

---

Future Work

Simulation: D-413 now provides the first runnable stationary responsive Ground background, displacement/recovery, curvature-well, orbital-restoring, and torque-ablation laboratory. It remains a reduced pre-quark model. Simulation: treat a 2D circulation only as a precursor; a Vortex Phase requires the native 3D D-409 layer. Simulation: roll-off event — measure spin tendency, pressure cushion, pressure pulse. Visualization: render actual lattice displacement, pressure, circulation, boundary strain, and failure from the same state arrays; do not substitute decorative animation. Experiment: Wave Reader measurement of stable oscillation. Derivation: complete charge emergence from B-06b pressure cushion. Derivation: complete the C-318 four-interaction carried-pattern response and absolute-energy calibration program.

Closing Thoughts

The question "what is a particle made of" dissolves when particles are redefined as patterns. A ripple on water is not made of water in any special sense. It is the water doing something. The ripple has no inside.

This reframing does not weaken physics. It removes the need for dozens of arbitrarily assigned parameters. Mass, charge, and spin are not properties bolted onto a point. They are geometric consequences of how the field behaves at its own boundary.

The Standard Model assigns these properties and gets extremely precise predictions. One-Wave asks where the properties come from and finds them in the geometry. Both are doing physics. They are asking different questions.

State Changer Plugin — Chapter 1

```
"""
State Changer Plugin v1.1
```

```python
Embedded in: Book 1, Chapter 1 — What Is a Particle?

Purpose:
    Register this chapter and its key items into the repository database.
    Track state changes for each item as the framework develops.
    Log emergence events when proofs or derivations complete.

Usage:
    Run this block directly to seed the chapter into the database.
    db_path can be pointed at your central One-Wave repository database.
"""

import sqlite3
from datetime import datetime, timezone

PLUGIN_NAME = "State Changer Plugin"
PLUGIN_VERSION = "1.1.0"

VALID_DIRECTIONS = {"expressive", "compressive", "mutual"}
VALID_EFFECTS = {"phase_shift", "modulate", "settle", "reset"}
VALID_STATUSES = {"proposed", "accepted", "rejected", "held", "modulated", "settled", "reset"}


def utc_now():
    return datetime.now(timezone.utc).isoformat()


def connect(db_path):
    conn = sqlite3.connect(db_path)
    conn.execute("PRAGMA foreign_keys = ON")
    return conn


def install_schema(cur):
    cur.execute("""
    CREATE TABLE IF NOT EXISTS plugin_meta (
        id INTEGER PRIMARY KEY,
        name TEXT NOT NULL,
        version TEXT NOT NULL,
        installed_at TEXT NOT NULL,
        notes TEXT
    )
    """)
    cur.execute("""
    CREATE TABLE IF NOT EXISTS node_registry (
        id INTEGER PRIMARY KEY,
        node_number INTEGER UNIQUE,
        node_key TEXT UNIQUE,
        title TEXT NOT NULL,
        appendix TEXT,
        original_node_id TEXT,
        status TEXT,
        description TEXT,
        created_at TEXT,
        updated_at TEXT
    )
    """)
```

```
cur.execute("""
CREATE TABLE IF NOT EXISTS node_links (
    id INTEGER PRIMARY KEY,
    from_node_id INTEGER NOT NULL,
    to_node_id INTEGER NOT NULL,
    relation TEXT,
    description TEXT,
    bidirectional INTEGER DEFAULT 1,
    created_at TEXT,
    FOREIGN KEY(from_node_id) REFERENCES node_registry(id),
    FOREIGN KEY(to_node_id) REFERENCES node_registry(id)
)
""")
cur.execute("""
CREATE TABLE IF NOT EXISTS books (
    id INTEGER PRIMARY KEY,
    book_number INTEGER UNIQUE,
    title TEXT NOT NULL,
    description TEXT,
    status TEXT,
    created_at TEXT,
    updated_at TEXT
)
""")
cur.execute("""
CREATE TABLE IF NOT EXISTS chapters (
    id INTEGER PRIMARY KEY,
    book_id INTEGER NOT NULL,
    chapter_number INTEGER,
    title TEXT NOT NULL,
    description TEXT,
    status TEXT,
    created_at TEXT,
    updated_at TEXT,
    FOREIGN KEY(book_id) REFERENCES books(id)
)
""")
cur.execute("""
CREATE TABLE IF NOT EXISTS chapter_items (
    id INTEGER PRIMARY KEY,
    chapter_id INTEGER NOT NULL,
    item_key TEXT,
    name TEXT NOT NULL,
    item_type TEXT,
    standard_model_definition TEXT,
    two_d_interpretation TEXT,
    three_d_interpretation TEXT,
    math_proof TEXT,
    state TEXT,
    description TEXT,
    created_at TEXT,
    updated_at TEXT,
    FOREIGN KEY(chapter_id) REFERENCES chapters(id)
)
""")
cur.execute("""
CREATE TABLE IF NOT EXISTS node_book_links (
```

```
    id INTEGER PRIMARY KEY,
    node_id INTEGER NOT NULL,
    book_id INTEGER,
    chapter_id INTEGER,
    item_id INTEGER,
    relation TEXT,
    description TEXT,
    created_at TEXT,
    FOREIGN KEY(node_id) REFERENCES node_registry(id),
    FOREIGN KEY(book_id) REFERENCES books(id),
    FOREIGN KEY(chapter_id) REFERENCES chapters(id),
    FOREIGN KEY(item_id) REFERENCES chapter_items(id)
)
""")
cur.execute("""
CREATE TABLE IF NOT EXISTS fields (
    id INTEGER PRIMARY KEY,
    node_id INTEGER,
    name TEXT NOT NULL,
    domain TEXT,
    state TEXT,
    description TEXT,
    created_at TEXT,
    updated_at TEXT,
    FOREIGN KEY(node_id) REFERENCES node_registry(id)
)
""")
cur.execute("""
CREATE TABLE IF NOT EXISTS field_choices (
    id INTEGER PRIMARY KEY,
    field_id INTEGER NOT NULL,
    name TEXT NOT NULL,
    direction TEXT NOT NULL,
    effect TEXT NOT NULL,
    definition TEXT,
    active INTEGER DEFAULT 1,
    created_at TEXT,
    updated_at TEXT,
    FOREIGN KEY(field_id) REFERENCES fields(id)
)
""")
cur.execute("""
CREATE TABLE IF NOT EXISTS state_changes (
    id INTEGER PRIMARY KEY,
    field_id INTEGER NOT NULL,
    from_state TEXT,
    to_state TEXT,
    choice_id INTEGER NOT NULL,
    bidirectional INTEGER DEFAULT 1,
    status TEXT NOT NULL DEFAULT 'proposed',
    validation_notes TEXT,
    repository_notes TEXT,
    created_at TEXT,
    updated_at TEXT,
    FOREIGN KEY(field_id) REFERENCES fields(id),
    FOREIGN KEY(choice_id) REFERENCES field_choices(id)
)
```

```python
    """)
    cur.execute("""
    CREATE TABLE IF NOT EXISTS validation_log (
        id INTEGER PRIMARY KEY,
        state_change_id INTEGER,
        node_id INTEGER,
        validator TEXT,
        result TEXT NOT NULL,
        notes TEXT,
        created_at TEXT,
        FOREIGN KEY(state_change_id) REFERENCES state_changes(id),
        FOREIGN KEY(node_id) REFERENCES node_registry(id)
    )
    """)
    cur.execute("""
    CREATE TABLE IF NOT EXISTS emergence_log (
        id INTEGER PRIMARY KEY,
        node_id INTEGER,
        field_id INTEGER,
        event_name TEXT NOT NULL,
        description TEXT,
        narrative_text TEXT,
        created_at TEXT,
        FOREIGN KEY(node_id) REFERENCES node_registry(id),
        FOREIGN KEY(field_id) REFERENCES fields(id)
    )
    """)
    cur.execute("""
    INSERT INTO plugin_meta (name, version, installed_at, notes)
    SELECT ?, ?, ?, ?
    WHERE NOT EXISTS (
        SELECT 1 FROM plugin_meta WHERE name=? AND version=?
    )
    """, (
        PLUGIN_NAME, PLUGIN_VERSION, utc_now(),
        "v1.1 Book 1 Chapter 1 seed.",
        PLUGIN_NAME, PLUGIN_VERSION,
    ))


def seed_book1_chapter1(db_path="onewave_repository.sqlite"):
    conn = connect(db_path)
    cur = conn.cursor()
    install_schema(cur)
    now = utc_now()

    # Register Book 1
    cur.execute("""
    INSERT INTO books (book_number, title, description, status, created_at, updated_at)
    VALUES (1, 'Micro', 'Compressed to One — Cell, Biology, Foundation Objects', 'active', ?, ?)
    ON CONFLICT(book_number) DO UPDATE SET updated_at=excluded.updated_at
    """, (now, now))
    cur.execute("SELECT id FROM books WHERE book_number=1")
    book_id = cur.fetchone()[0]

    # Register Chapter 1
    cur.execute("""
```

```
INSERT INTO chapters (book_id, chapter_number, title, description, status, created_at, updated
VALUES (?, 1, 'What Is a Particle?',
'No particles — only excitations. Spin, mass, charge from geometry.',
'GREEN', ?, ?)
""", (book_id, now, now))
chapter_id = cur.lastrowid

# Chapter items
items = [
    {
        "key": "B1C1-EXCITATION",
        "name": "Excitation",
        "type": "core_concept",
        "gray": "Standard Model: particle = discrete irreducible unit with assigned properties
        "two_d": "A persistent displacement pattern in the 2D field. Not a thing. A stable rip
        "three_d": "A Persistent Mode of the superfluid lattice field psi. No interior, no bou
        "math": "||psi_{n+k} - psi_n|| < epsilon",
        "state": "GREEN",
        "desc": "Replaces 'particle' in One-Wave. Everything is an excitation."
    },
    {
        "key": "B1C1-SPIN",
        "name": "Spin-half from Roll-Off",
        "type": "derived_property",
        "gray": "Standard Model: spin is an intrinsic quantum number with no classical analog.
        "two_d": "2D field mode presses against boundary, surface tension holds, rolls off asy
        "three_d": "Roll-off at mode boundary produces 4*pi closure. psi(theta+2pi)=-psi(theta
        "math": "DeltaE = E_mode - sigma*A_s. DeltaE > 0 and S_roll != 0 => spin tendency.",
        "state": "YELLOW",
        "desc": "Spin emerges from boundary roll-off geometry. Formal proof incomplete (B-06b)
    },
    {
        "key": "B1C1-MASS",
        "name": "Mass Effect from Four-Interaction Response",
        "type": "derived_property",
        "gray": "Standard physics measures inertia and uses Higgs/strong-interaction language
        "two_d": "A bounded recurrence resists having its complete pattern relocated relative
        "three_d": "Knot, electrical shell, Mirror relation, Boundary-Tension Weave, and cross
        "math": "M_ij = d^2 E4 / (dv_i dv_j) at v=0; m_eff = Tr(M)/3 for an isotropic mode.",
        "state": "YELLOW",
        "desc": "False transport-to-mass shortcut and scalar-gap import removed; four-interact
    },
    {
        "key": "B1C1-CHARGE",
        "name": "Charge from Pressure Cushion",
        "type": "derived_property",
        "gray": "Standard Model: electric charge is an intrinsic quantum number. Quantized in
        "two_d": "Roll-off produces a pressure cushion in the background field. Cushion is the
        "three_d": "E_vec ~ nabla P_cushion (radial = electric). B_vec ~ nabla x P_cushion (ro
        "math": "P_c = beta * DeltaE / V_c. E_vec ~ nabla P_c. B_vec ~ nabla x P_c.",
        "state": "YELLOW",
        "desc": "Charge from pressure cushion at boundary. Complete derivation deferred (B-06b
    },
    {
        "key": "B1C1-HIGGS",
        "name": "125 GeV Mirror-Gate Boundary Response",
        "type": "prediction",
```

```
            "gray": "Standard Model: the approximately 125 GeV collider response is interpreted as
            "two_d": "The coupled boundary must be pushed through a finite Mirror-flip threshold."
            "three_d": "A-115/C-322 interpret 125 GeV as pressure-work across the knot, shell, Mir
            "math": "E_MG = E4(q_G)-E4(q_0) = integral_Gamma grad(E4) dot dq ~= 125 GeV.",
            "state": "YELLOW",
            "desc": "The measurement is retained; the separate-particle interpretation is challeng
        },
    ]

    for item in items:
        cur.execute("""
        INSERT INTO chapter_items
        (chapter_id, item_key, name, item_type,
         standard_model_definition, two_d_interpretation, three_d_interpretation,
         math_proof, state, description, created_at, updated_at)
        VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)
        """, (
            chapter_id, item["key"], item["name"], item["type"],
            item["gray"], item["two_d"], item["three_d"],
            item["math"], item["state"], item["desc"], now, now
        ))
        item_id = cur.lastrowid

        # Register dependency nodes and link to chapter item
        dep_map = {
            "B1C1-EXCITATION": [(101, "A-10", "Appendix A", "Persistent Mode", "GREEN"),
                                 (109, "A-09", "Appendix A", "Recursion", "YELLOW")],
            "B1C1-SPIN":       [(106, "B-06b", "Appendix B", "Four Views Roll-Off", "YELLOW"),
                                 (108, "C-08", "Appendix C", "Spin-half", "GREEN")],
            "B1C1-MASS":       [(318, "C-318", "Appendix C", "Mass-Effect Mechanism Resolution", "
            "B1C1-CHARGE":     [(106, "B-06b", "Appendix B", "Four Views Roll-Off", "YELLOW"),
                                 (110, "C-311", "Appendix I", "Electric-Magnetic Duality", "GREEN"
            "B1C1-HIGGS":      [(322, "C-322", "Appendix C", "Mirror-Gate 125 GeV Boundary Respons
        }

        for node_num, orig_id, appendix, title, status in dep_map.get(item["key"], []):
            cur.execute("""
            INSERT INTO node_registry
            (node_number, node_key, title, appendix, original_node_id, status, description, create
            VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?)
            ON CONFLICT(node_number) DO UPDATE SET updated_at=excluded.updated_at
            """, (node_num, f"Node_{node_num}", title, appendix, orig_id, status,
                  f"Dependency for {item['key']}", now, now))
            cur.execute("SELECT id FROM node_registry WHERE node_number=?", (node_num,))
            node_db_id = cur.fetchone()[0]

            cur.execute("""
            INSERT INTO node_book_links
            (node_id, book_id, chapter_id, item_id, relation, description, created_at)
            VALUES (?, ?, ?, ?, 'supports', ?, ?)
            """, (node_db_id, book_id, chapter_id, item_id,
                  f"{orig_id} supports {item['key']}", now))

    # Emergence log entry
    cur.execute("""
    INSERT INTO emergence_log (event_name, description, narrative_text, created_at)
    VALUES (
```

```
        'Book 1 Chapter 1 Seeded',
        'Chapter 1 What Is a Particle registered with 5 items: Excitation, Spin, Mass, Charge, Hig
        'Void allowed field. Field allowed choice. Choice allowed particle to dissolve into excita
        ?
    )
    """, (now,))

    conn.commit()
    conn.close()
    print("Book 1 Chapter 1 seeded into database.")
    print(f"  Book ID: {book_id}")
    print(f"  Chapter ID: {chapter_id}")
    print(f"  Items registered: {len(items)}")


if __name__ == "__main__":
    seed_book1_chapter1()
```

END OF BOOK 1 CHAPTER 1 One wave. Mirror builds. Mark Wright. Kitty Hawk V0.